

Shiv Nadar University Chennai

Name	Joshua	Roll Number	24110085
Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester	V
Subject Code & Name	CS3807 - Deep Learning Laboratory	AY	2026-27
Batch	2	Experiment Date	09 July 2026

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1 Objective

To build a Single Layer Perceptron from scratch and use it to classify banknotes as authentic or forged. The experiment also studies the data, training error, learning rate, normalization, and model performance.

2 Background Theory

A perceptron is one artificial neuron. It multiplies each input by a weight, adds a bias, and uses an activation function.

$$z = \mathbf{w}^T \mathbf{x} + b$$

$$\hat{y} = f(z)$$

For a wrong prediction, the parameters are updated as follows:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} + \eta(y - \hat{y})\mathbf{x}$$

$$b_{\text{new}} = b_{\text{old}} + \eta(y - \hat{y})$$

Here, η is the learning rate. A single perceptron can learn only a linear decision boundary.

3 Activation Function

The experiment uses the Step function: $f(z) = 1$ for $z \geq 0$ and 0 for $z < 0$. It gives a hard class output. Sigmoid gives a smooth output between 0 and 1 , so it is more useful in networks trained with backpropagation.

Activation	Output	Common use
Step	0 or 1	Classical perceptron
Sigmoid	0 to 1	Binary output layer
Tanh	-1 to 1	Hidden layers
ReLU	0 to ∞	Modern hidden layers
Leaky ReLU	$-\infty$ to ∞	Avoiding dying ReLU
Softmax	Probabilities	Multi-class output

4 Dataset

The Banknote Authentication dataset was used. It contains 1372 rows, four numerical features, two classes, and no missing values.

Item	Value
Source	UCI Machine Learning Repository
Features	Variance, Skewness, Curtosis, Entropy
Class 0	Authentic banknote
Class 1	Forged banknote
Training rows	1097
Testing rows	275

4.1 First Five Samples

Variance	Skewness	Curtosis	Entropy	Class
3.6216	8.6661	-2.8073	-0.44699	0
4.5459	8.1674	-2.4586	-1.4621	0
3.866	-2.6383	1.9242	0.10645	0
3.4566	9.5228	-4.0112	-3.5944	0
0.32924	-4.4552	4.5718	-0.9888	0

4.2 Descriptive Statistics

Statistic	Variance	Skewness	Curtosis	Entropy	Class
count	1372.0000	1372.0000	1372.0000	1372.0000	1372.0000
mean	0.4337	1.9224	1.3976	-1.1917	0.4446
std	2.8428	5.8690	4.3100	2.1010	0.4971
min	-7.0421	-13.7731	-5.2861	-8.5482	0.0000
25%	-1.7730	-1.7082	-1.5750	-2.4134	0.0000
50%	0.4962	2.3197	0.6166	-0.5866	0.0000
75%	2.8215	6.8146	3.1792	0.3948	1.0000
max	6.8248	12.9516	17.9274	2.4495	1.0000

5 Procedure

First, I checked the dataset and looked for missing values. I then made graphs to understand the features. The data was divided into training and testing sets. Only the training data was used to fit the scaler. After that, I trained the perceptron and checked its performance using the required metrics. I also tested different learning rates, raw data, standardized data, and the Scikit-learn model.

6 Source Code

<https://github.com/Joshua-Divide/LAB>

7 Generated Plots

7.1 Feature Histograms

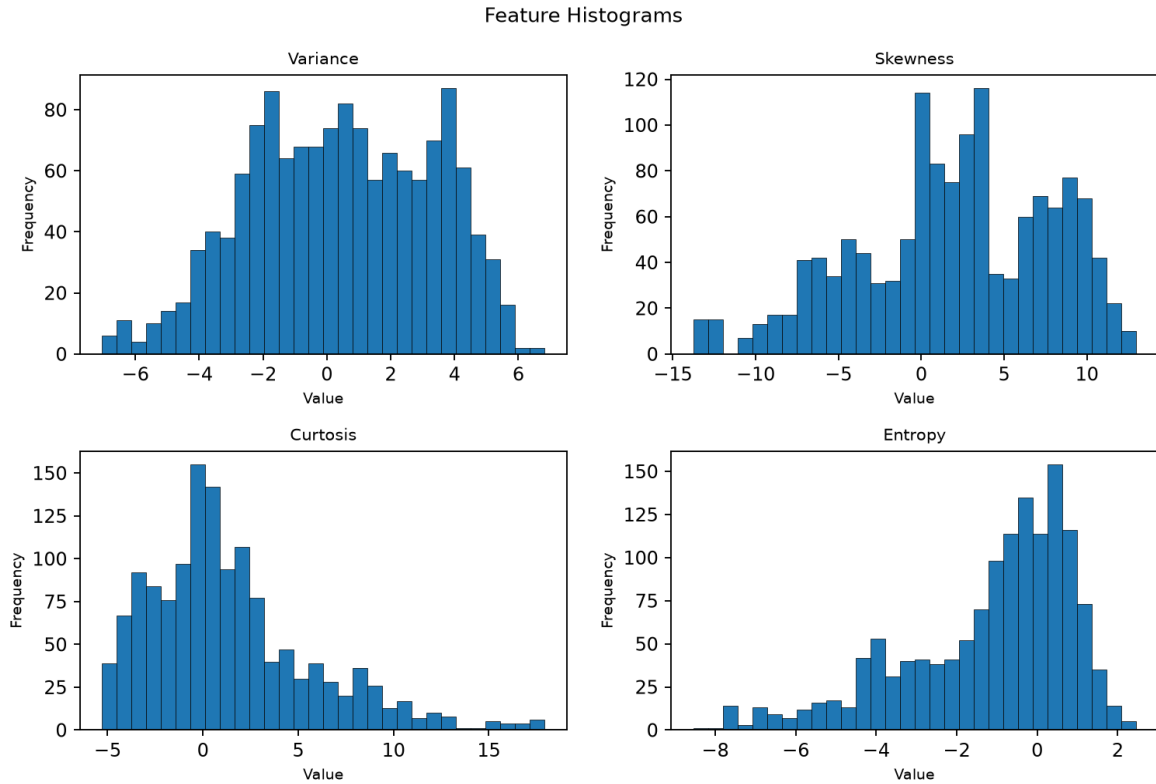


Figure 1: Feature histograms

The four features have visibly different ranges and distribution shapes. Kurtosis has a pronounced right tail, while the other features show different degrees of asymmetry and spread.

7.2 Correlation Heatmap

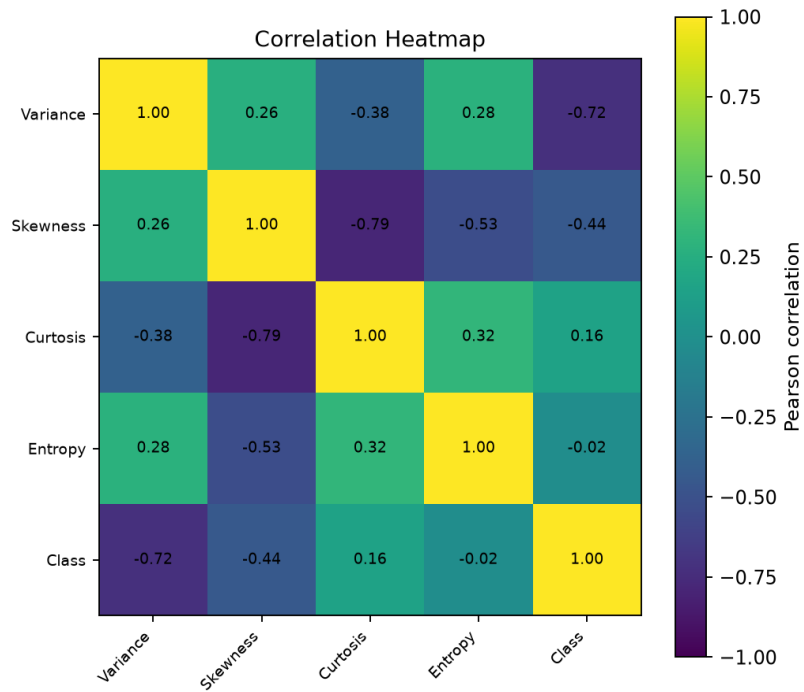


Figure 2: Correlation heatmap

Variance has a strong negative correlation with the class (-0.72), while skewness is also related to the class (-0.44). Skewness and kurtosis show a strong negative feature-to-feature relationship (-0.79).

7.3 Scatter Plot

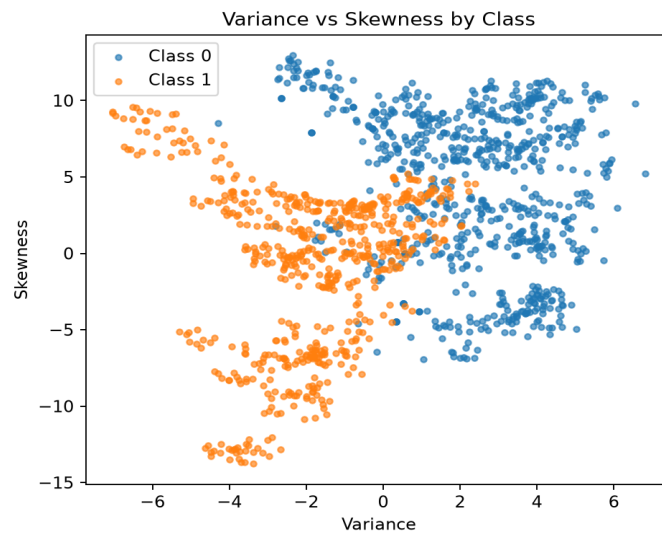


Figure 3: Variance and skewness by class

Most points from the two classes occupy different regions, but there is overlap in this two-dimensional projection. Therefore, variance and skewness alone do not provide perfect linear separation.

7.4 Feature Boxplots

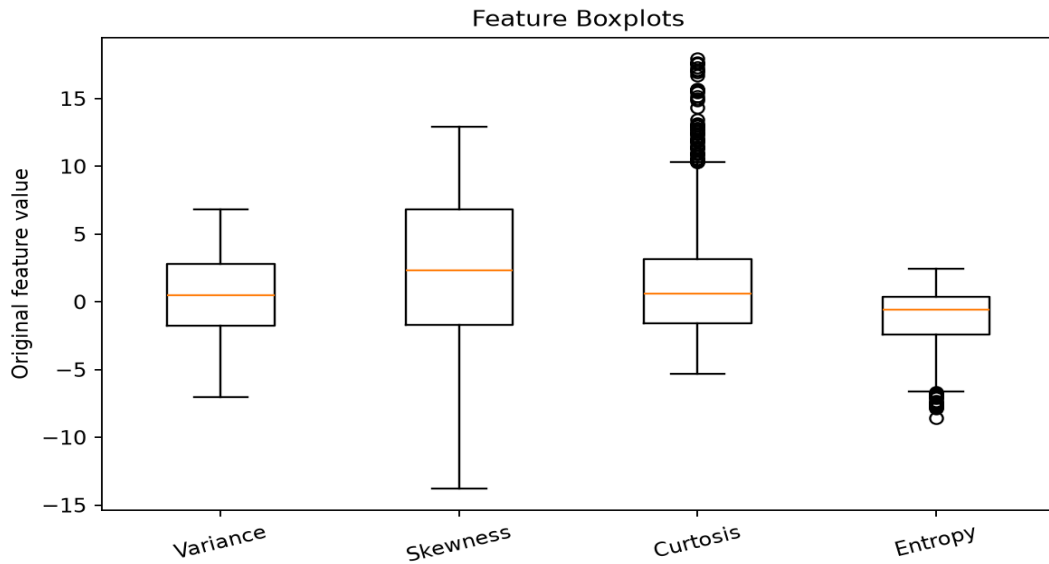


Figure 4: Feature boxplots

The boxplots show that the four features have different spreads and contain several extreme values. Standardization places their numerical scales on a comparable basis.

7.5 Training Error

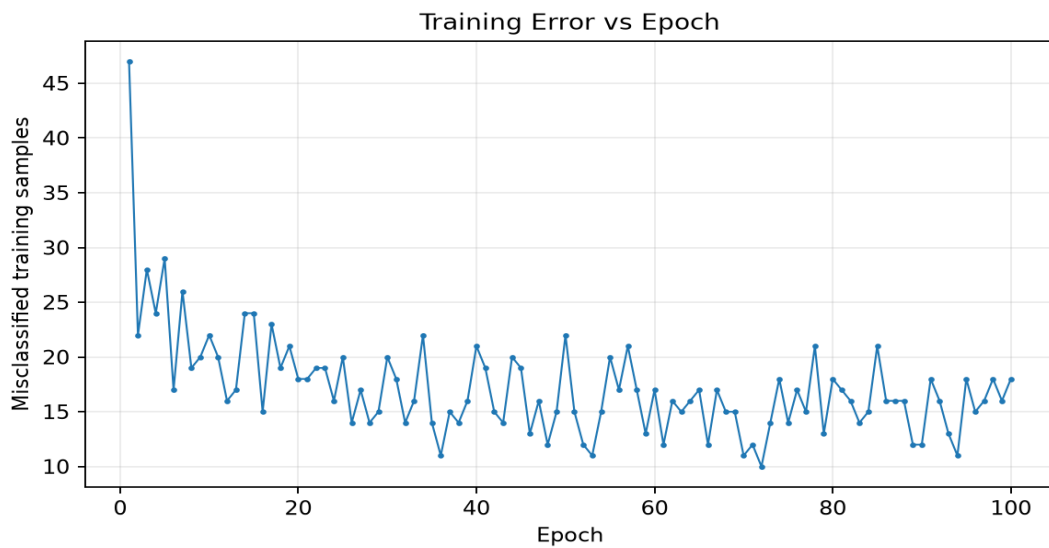


Figure 5: Training error against epoch

The number of mistakes drops sharply early in training and then fluctuates. The model ends with 18 misclassified training samples after 100 epochs.

7.6 Weight Evolution

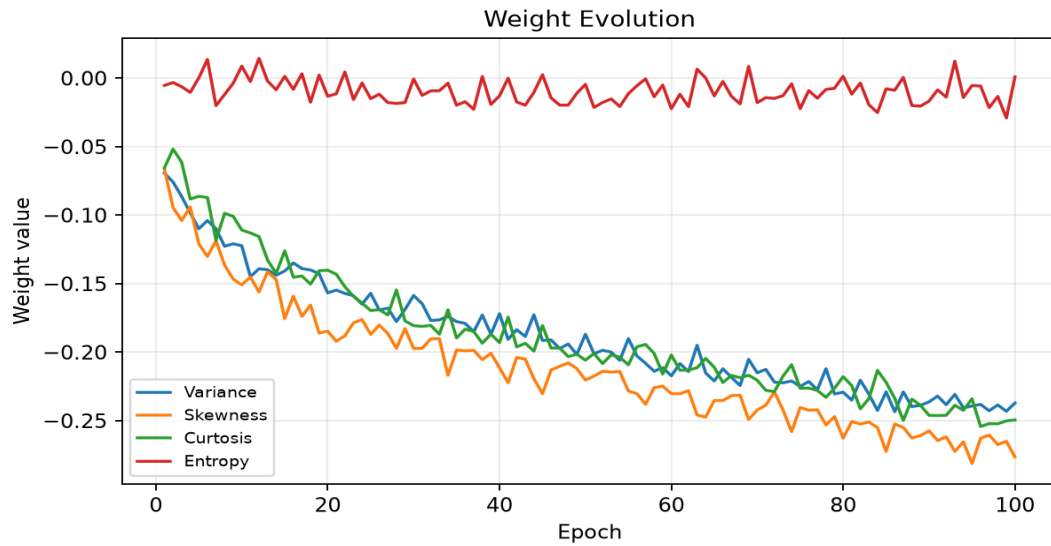


Figure 6: Weight evolution

The weights change most strongly during the earlier updates as the perceptron adjusts its linear boundary. Later changes are smaller, although the non-zero training error prevents complete stabilization.

7.7 Bias Evolution

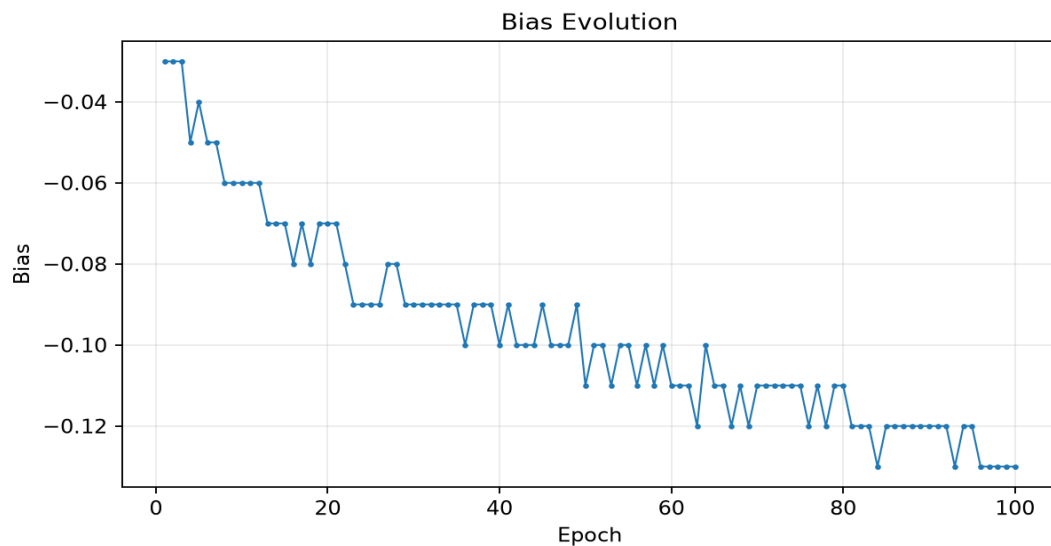


Figure 7: Bias evolution

The bias changes along with the weights and shifts the learned decision hyperplane away from the origin.

7.8 Confusion Matrix

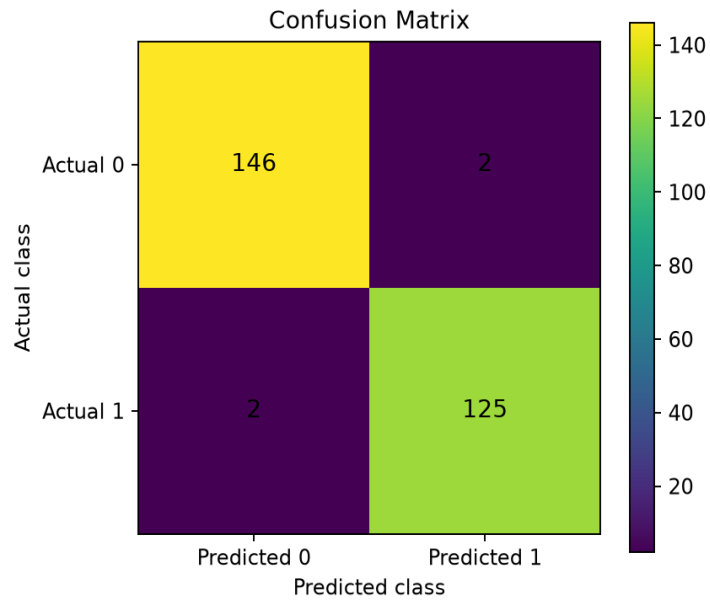


Figure 8: Confusion matrix

Using forged banknotes as the positive class: TN = 146, FP = 2, FN = 2, and TP = 125.

7.9 Learning Rate Comparison



Figure 9: Learning-rate comparison

Learning rates 0.001, 0.01, and 0.1 were tested using the same initialization and training order. Their convergence curves can be compared directly from the plot.

7.10 Normalization Comparison

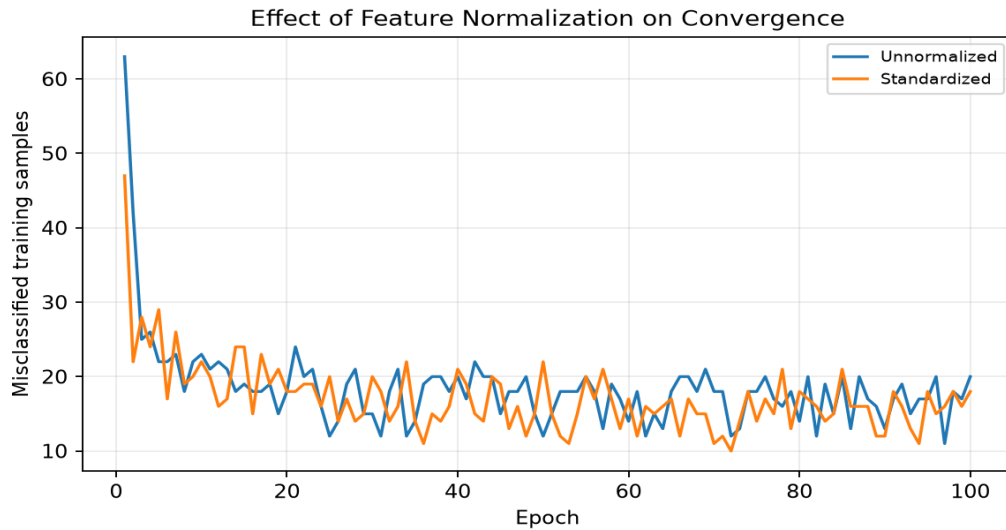


Figure 10: Raw and standardized training

The raw model ends with 20 training errors, while the standardized model ends with 18. Standardization also changes the convergence trajectory.

7.11 Decision Boundary

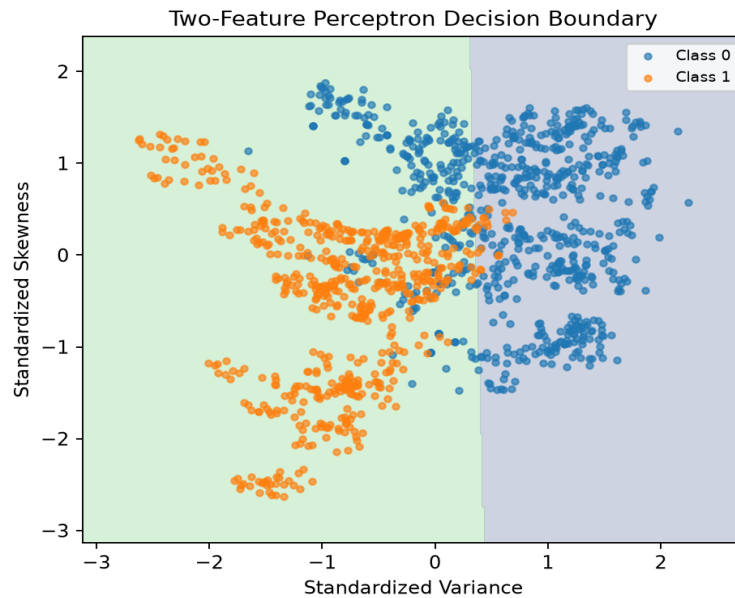


Figure 11: Two-feature decision boundary

The two-feature model produces a straight decision line because a single perceptron has a linear boundary. This visualization uses only variance and skewness, so it omits information from kurtosis and entropy.

7.12 Logic Gate Visualization with a Single-Layer Perceptron

AND Gate - Boundary from Initialization to Convergence

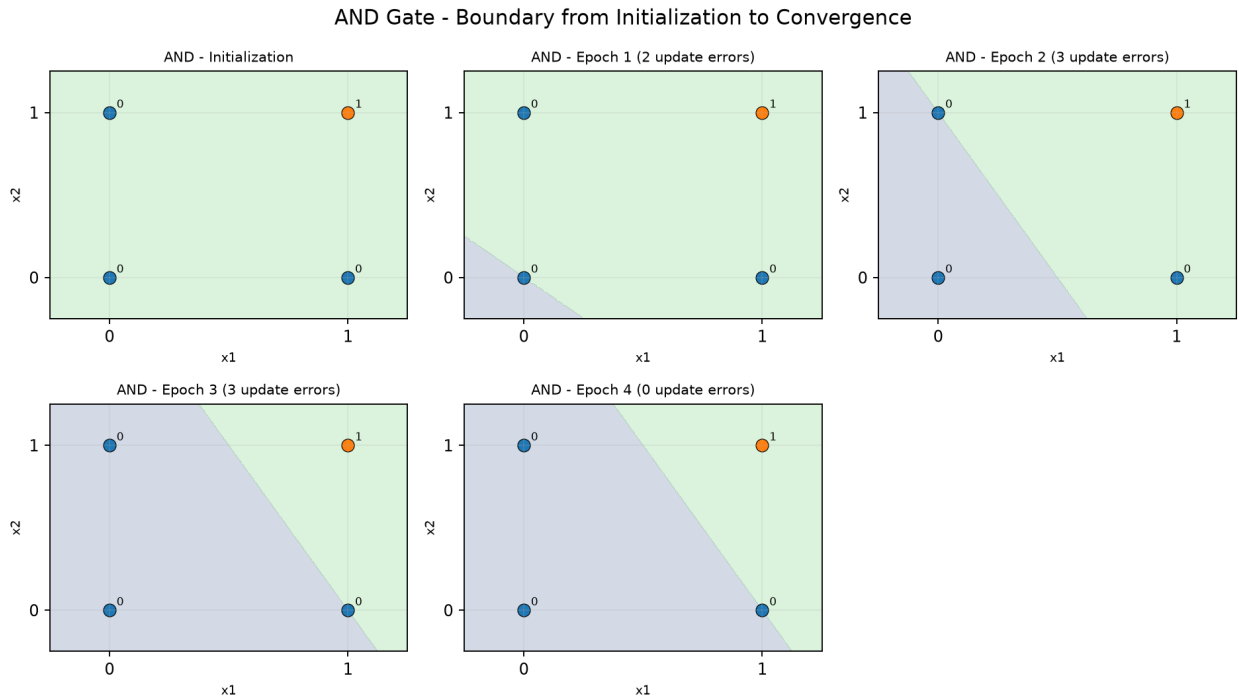


Figure 12: AND gate from initialization through every epoch until convergence

The AND truth table is linearly separable. Starting from zero weights and zero bias, the perceptron updates its boundary and reaches an epoch with zero update errors.

OR Gate - Boundary from Initialization to Convergence

OR Gate - Boundary from Initialization to Convergence

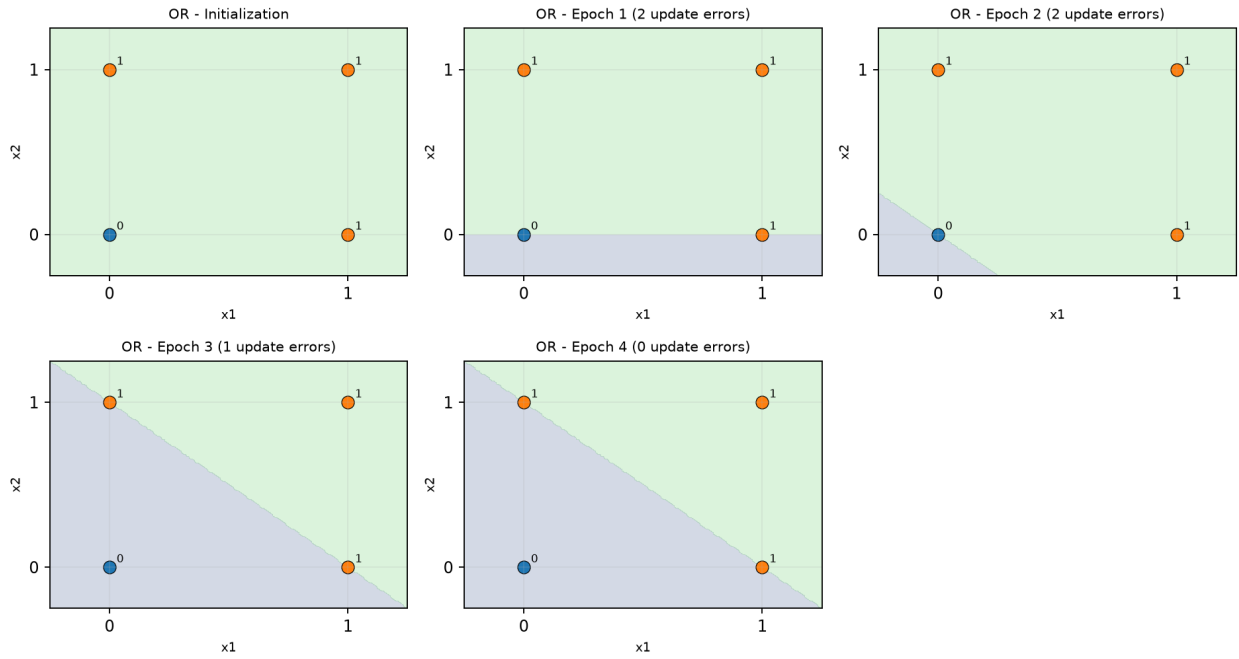


Figure 13: OR gate from initialization through every epoch until convergence

The OR truth table is also linearly separable. The sequence shows every epoch from initialization until the perceptron finds a separating boundary with zero update errors.

XOR Gate - Consecutive Perceptron Updates

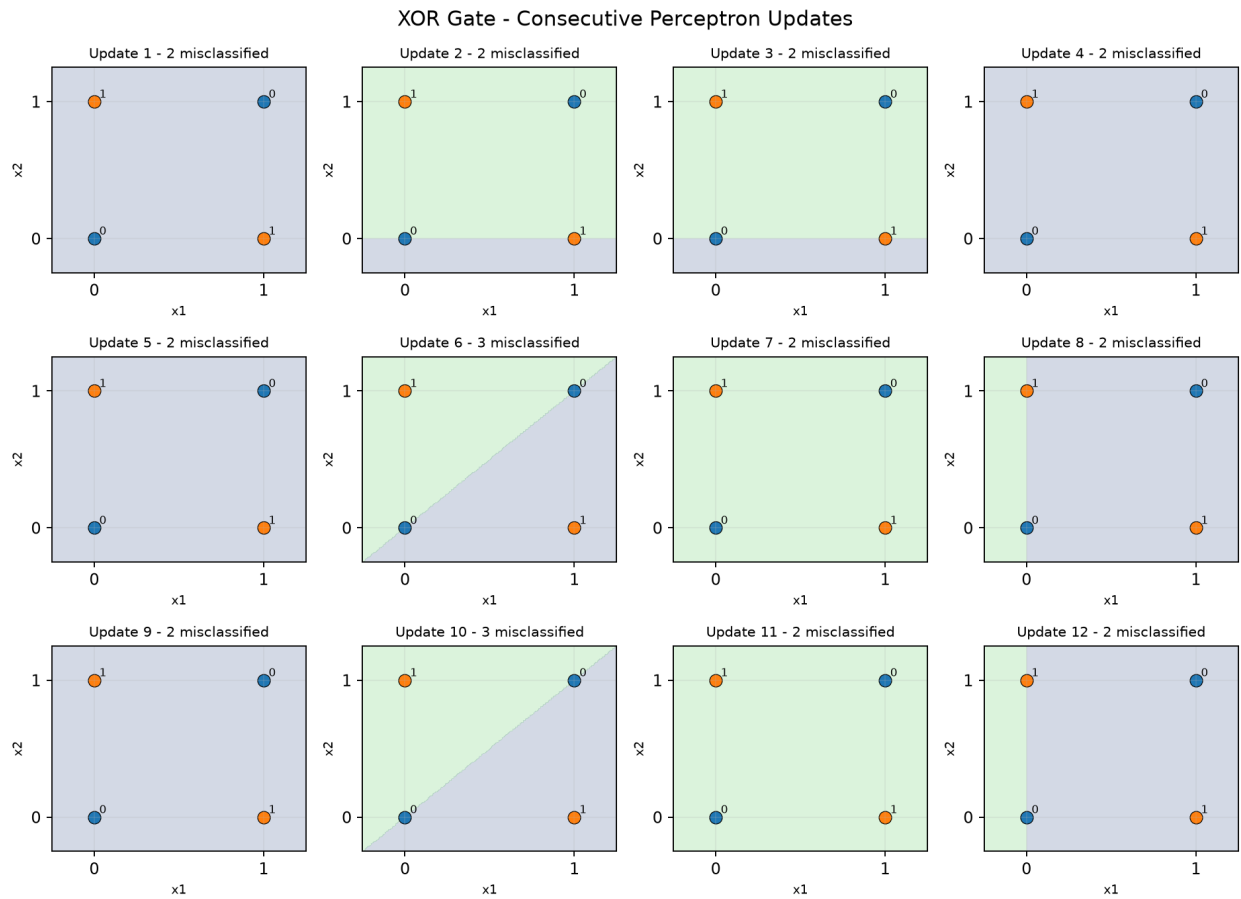


Figure 14: Twelve consecutive XOR perceptron updates, arranged four per row

XOR is not linearly separable. The boundary keeps changing as the perceptron tries to satisfy conflicting samples, so a single linear decision boundary cannot classify all four XOR points correctly.

7.13 Step and Sigmoid

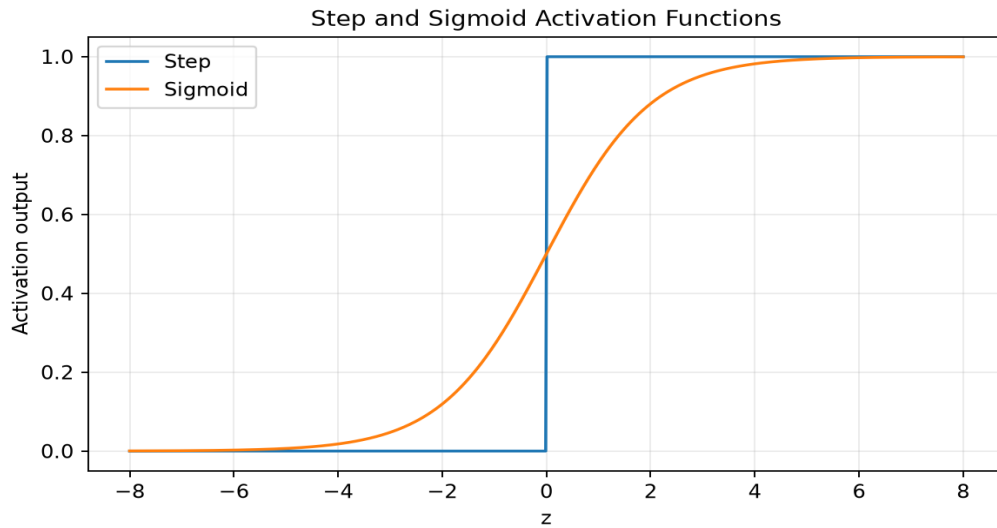


Figure 15: Step and Sigmoid functions

The Step function changes abruptly from one class output to the other. Sigmoid changes smoothly and has a derivative, which makes gradient-based backpropagation possible.

8 Performance Tables

8.1 Training Summary

Item	Value
Dataset size	1372
Train/Test split	1097 / 275
Learning rate	0.01
Maximum epochs	100
Epochs run	100
Final training errors	18
Final weights	[-0.237046, -0.276396, -0.249419, 0.000965]
Final bias	-0.130000
Accuracy	0.985455
Precision	0.984252
Recall	0.984252
F1-score	0.984252

8.2 First Five Epochs

Epoch	Errors	Bias	w1	w2	w3	w4
1	47	-0.0300	-0.0692	-0.0663	-0.0653	-0.0052
2	22	-0.0300	-0.0758	-0.0944	-0.0517	-0.0031
3	28	-0.0300	-0.0863	-0.1039	-0.0613	-0.0062
4	24	-0.0500	-0.0982	-0.0939	-0.0882	-0.0104
5	29	-0.0400	-0.1098	-0.1210	-0.0862	0.0003

8.3 Learning Rates

Rate	Errors	Accuracy	Precision	Recall	F1
0.001	18	0.985455	0.984252	0.984252	0.984252
0.010	18	0.985455	0.984252	0.984252	0.984252
0.100	18	0.985455	0.984252	0.984252	0.984252

8.4 Scratch and Scikit-learn

Model	Accuracy	Precision	Recall	F1
Scratch	0.985455	0.984252	0.984252	0.984252
Scikit-learn	0.985455	0.984252	0.984252	0.984252

8.5 Normalization Result

Data	Errors	Accuracy	Precision	Recall	F1
Raw	20	0.981818	0.976562	0.984252	0.980392
Standardized	18	0.985455	0.984252	0.984252	0.984252

9 Additional Questions

9.1 Why is Sigmoid better for deep learning?

Sigmoid changes smoothly and has a derivative. This allows gradients to be propagated through a network during backpropagation, whereas the Step function is not suitable for gradient-based training.

9.2 What is the effect of learning rate?

The learning rate controls the size of each perceptron update. Smaller values create smaller parameter changes and larger values create larger changes; the plotted experiments compare their convergence behavior.

9.3 Why can a single perceptron not solve XOR?

XOR places the positive and negative examples on opposite corners of the input square. No single straight line can separate the two classes, so a hidden layer or another non-linear model is required.

9.4 Why does normalization help?

The perceptron update uses feature values directly. Standardization makes the feature magnitudes comparable, preventing large-scale features from dominating individual parameter updates and often improving convergence behavior.

10 Discussion

The scratch perceptron reached 98.55% test accuracy in this run. It still produced 18 training errors after 100 epochs, showing that it did not reach zero training error within the chosen training duration. The confusion matrix and normalization comparison show how the learned classifier behaves on the held-out data. The AND and OR demonstrations converge because their truth tables are linearly separable, while the XOR update sequence demonstrates the limitation of a single linear perceptron.

11 Conclusion

The Single Layer Perceptron was implemented from scratch and evaluated on the Banknote Authentication dataset. It achieved an accuracy of 0.985455 and an F1-score of 0.984252 in this run. The experiment demonstrates the role of weights, bias, learning rate, feature scaling, and linear separability. The logic-gate visualizations reinforce that a single perceptron can learn AND and OR but cannot represent XOR.

12 References

1. F. Rosenblatt, "The Perceptron," Psychological Review, 1958.
2. UCI Machine Learning Repository, Banknote Authentication Dataset.
3. Scikit-learn Documentation, Perceptron and StandardScaler.
4. I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning, MIT Press, 2016.